■ Genetic Algorithm + Machine Learning Timetable Optimization System

Institution: Dwarkadas J. Sanghvi College of Engineering

Department: Data Science

Semester: IV

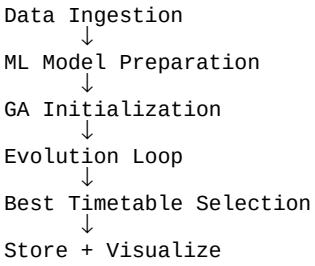
Divisions: D1, D2, D3

Architecture: Genetic Algorithm + Machine Learning + Database + Frontend Integration

■ 1. Overall System Architecture

This system is a hybrid intelligent scheduling engine designed to generate optimized academic timetables aligned with NEP-based flexibility and institutional constraints.

The architecture integrates constraint satisfaction, evolutionary optimization, and predictive machine learning models to generate high-quality academic schedules.



■ 2. Block 1 – Data Ingestion and Preprocessing

This block prepares the structural foundation of the scheduling problem. All academic and infrastructure metadata is loaded and transformed into in-memory models before optimization begins.

2.1 Administrative Inputs

• Subjects and subject codes

• Credits and weekly instructional hours

• Subject type classification (theory, lab, elective, project)

• Faculty mapping and availability

• Classroom and laboratory metadata

• Time slots and break configuration

• Constraint thresholds and NEP alignment parameters

2.2 Preprocessing Structures

```
// Core Schedule Structure
schedule[division][day][slot]

// Faculty Availability
facultyAvailability[facultyId][day][slot]

// Room Availability
roomAvailability[roomId][day][slot]

// Weekly Subject Requirements
requiredHours[division][subjectId]
```

All these structures are combined into a unified context object used by both GA and ML layers.

■ 3. Block 2 – Machine Learning Model Preparation

Machine Learning models provide predictive intelligence that enhances the quality of timetable optimization. These models do not generate schedules but influence fitness evaluation.

3.1 Difficulty Load Model

Predicts subject difficulty based on historical academic metrics including performance trends and assessment weightage.

```
difficulty_score = difficultyModel.predict(subjectFeatures)
// Output range: 0 to 1
```

3.2 Cognitive Fatigue Model

Predicts daily student cognitive overload using sequence-aware academic features.

```
fatigue_index = fatigueModel.predict(dayFeatures)
// Output range: 0 to 1
```

3.3 Infrastructure Stress Model

Predicts laboratory congestion and infrastructure overload using dynamic weekly distribution features.

```
infra_stress_score = infraModel.predict(weekFeatures)
// Output range: 0 to 1
```

This model ensures laboratory sessions are distributed evenly across the week to prevent equipment strain and supervision overload.

■ 4. Block 3 – Initial Population Generation

The Genetic Algorithm begins by generating an initial population of candidate timetables.

Population Size: 80 Chromosomes (configurable)

4.1 Initialization Steps

Step 1: Allocate fixed break slots

Step 2: Allocate laboratory sessions with consecutive slot enforcement

Step 3: Allocate theory subjects respecting credit requirements

Step 4: Apply repair functions to eliminate initial violations

```
assignBreakSlots()
allocateLabBlocks()
allocateTheorySessions()
repairSchedule()
```

■ 5. Block 4 – GA Evolution Loop

The GA evolves the population over multiple generations to minimize fitness value.

5.1 Selection

```
function tournamentSelection(population) {
    const sample = pickRandom(3)
    return lowestFitness(sample)
}
```

5.2 Crossover

```
if (Math.random() < crossoverRate) {
    child = divisionWiseCrossover(parentA, parentB)
}
```

5.3 Mutation

```
if (Math.random() < mutationRate) {
    mutate(child)
}
```

5.4 Fitness Function

```
fitness =
    10000 * hardPenalty
  + 100   * softPenalty
  + 40    * difficultyPenalty
  + 60    * fatiguePenalty
  + 30    * infraPenalty
```

Hard constraints dominate scoring. ML penalties refine quality.

■ 6. Final Timetable Selection and Deployment

After termination criteria are met, the chromosome with minimum fitness is selected.

```
bestChromosome = population.reduce(minFitness)
```

The final timetable is then serialized and stored in MongoDB.

```
await Timetable.create(serialize(bestChromosome))
```

Frontend renders division-wise, faculty-wise, and room-wise views.

Optional modules include PDF export and analytics dashboards.

■ 7. System Summary

Database Layer: Stores academic metadata.

ML Layer: Predicts dynamic quality indicators.

GA Engine: Optimizes timetable structure.

Fitness Layer: Integrates predictive intelligence and constraint penalties.

Adaptive Module: Prevents stagnation and improves convergence.

Frontend Layer: Visualization and user interaction.

This hybrid system ensures conflict-free, cognitively balanced, and infrastructure-aware timetable generation suitable for institutional deployment.